School : Computer Science
Institution : University of Windsor
Term : Winter 2025
Course : Comp-3150-1: Database Management Systems
Instructor : Dr. C. I. Ezeife
Assignment #2 : Total : 50 marks
Handed Out: Thurs. Jan. 30, 2025; Due Thurs. Feb. 27, 2025

Objective of Assignment: To test on knowledge and design of relational model constraints, relational database schemas, functional dependencies and normalization of relational databases.

Scope: Assignment covers materials from Chapters 5 and 14 of book discussed in class.

Electronic Assignment Submission: Done through http://brightspace.uwindsor.ca

Marking Scheme : The mark for each of the questions is indicated beside each question.

Academic Integrity Statement : Remember to submit only work that is yours and include the following confidentiality agreement and statement at the beginning of your assignment.

CONFIDENTIALITY AGREEMENT & STATEMENT OF HONESTY

I confirm that I will keep the content of this assignment/examination confidential.
I confirm that I have not received any unauthorized assistance in preparing for or doing this assignment/examination. I confirm knowing that a mark of 0 may be assigned for copied work.

Dante LUCIANI — Student Signature
Dante Chang Luciani — Student Name (please print)
110127893 — Student I.D. Number
2/27/2025 — Date

Marking Scheme : The mark for each question and sub question is shown with the question below. Place your solutions in tables where possible.

For office Use only

Question	Mark
1	/20
2	/10
3	/10
4	/10
Total	/50

CHAPTER 5: THE RELATIONAL DATA MODEL AND RELATIONAL DATABASE CONSTRAINTS

1. (total marks 20) Given the same Given the simple flight_seat_Reservation database schema that contains four files (tables) described as follows, answer the following questions with regards to this database.

(Total for que 1 is 10 marks)

Airport(Airportcode, AName, City, State)
Flight(Flight#, Airline)
Fare(Flight#, farecode, Amount)
Reservation(Flight#, Fltdate, Seat#, fltDtime, fltAtime, fltDAport, fltAAport, Cname)

Note :
Airportcode is of type string (e.g., YYZ, YTZ, YQT, YQG, YYC). Aname is airport name of type string for instance the corresponding airport names for the codes above are : Pearson Airport, Island Airport, Thunder Bay Airport, Windsor Airport, Calgary Airport. While the first two named airports are in Toronto city, ON state/province, the last three airports are in the cities mentioned in their name respectively. Also, Flight# can be drawn from the following sample domain of string data type WJ250, WJ261, AC275, AC300, AC320, PA233 from three airlines of WestJet, Air Canada, Porter Airline respectively. Possible farecodes are F1, F2, F3 for each airline for providing different fares. It is up to you to decide what amount (e.g., $400.50) of fare to fix for those codes (there is no wrong answer with fixing the amount). The file (table) reservation is a relationship (activity) file relating the entities airport, flight and fare and providing details of unique seat number (e.g, 5A, 5B) assigned to a customer (Cname) for a flight on a date (fltdate, eg. 10-Jan-25) at a specific departure time (flighDtime, e.g., 8.00) and arrival time (fltAtime, e.g. 14.00) at a specific flight departure airport code (fltDAport) and flight arrival airport code (fltAAport) in the database.

Assume that an update operation (a general term in this chapter 5, for an insert, a modify or a delete operation) is to be made to this database to enter information about a new flight not already in the database but which has just been scheduled with an existing airline Air Canada, assigned flight dates and times and departing and arriving airports and with a customer waiting to reserve a seat. Answer the following questions on what specific relations, attributes and operations (eg. insert, modify, delete) that need to be done for this update to be implemented in the entire database. This is not SQL query yet.
Provide your answers both in descriptive sentence and using the formal (not SQL) database operations of INSERT, MODIFY, DELETE as used in Chapter 5 of book with specific attributes and relations when possible.
An example formal insert of a flight record into the Flight table is:
INSERT < Flight#, Airline> into Flight // for the new Flight record
And an example descriptive sentence for this insert of a flight record into the Flight table is: do an insert operation for the new flight record into the Flight table. (Note that the above is an example and you still need to update all other database tables that need to be updated to

reflect information about this new flight, such as tables Fare and Reservation).

(a) Give the set of needed insert, modify or delete operations for this update operation scenario described above. 5 marks
(b) What types of integrity constraints (explain using attributes, eg, Flight# of relevant files) would you expect to check for this update to be done? 5 marks
(c) Which of these integrity constraints are key, entity integrity, and referential (foreign key) integrity constraints and which are not? 5 marks
(d) Specify all the referential integrity (foreign key) constraints on this database in the format Referring_Relation.Attribute --> Referred_Relation.Attribute.
5 marks
(Total for que 1 is 20 marks)

From Assign1: An instance of the flight_seat_Reservation database is :

Airport Table

Airportcode	AName	City	State
YYZ	Pearson Airport	Toronto	ON
YTZ	Island Airport	Toronto	ON
YQT	Thunder Bay Airport	Thunder Bay	ON
YYC	Calgary Airport	Calgary	AB

Flight Table

Flight#	Airline
WJ250	WestJet
AC300	Air Canada
AC275	Air Canada
PA233	Porter Airline

Fare Table

Flight#	farecode	Amount
WJ250	F1	400.50
AC300	F2	500.50
AC275	F1	400.50
PA233	F3	450.50

Reservation Table

Flight#	Fltdate	Seat#	FltDtime	FltAtime	fltDAport	fltAAport	Cname
WJ250	10-Jan-25	5A	08:00	14:00	YYZ	YYC	John Smith

AC300	15-Feb-25	12B	09:30	12:00	YYC	YTZ	Joan Baez
AC275	16-Feb-25	4C	15:00	18:00	YTZ	YQT	Bob Dylan
PA223	05-Apr-25	1A	7:00	10:00	YYC	YYZ	Heath Ledger

Solution:

Question	**Answers**
a. Give the operations for this update. 5 marks	One possible set of operations for the given update is the following: 1. Insert the flight into flight table INSERT < 'AC293', 'Air Canada' > into Flight Perform an insert operation to add new flight into the Flight table with its assigned flight number and airline. 2. Insert fares for the newly added flight into Fare table INSERT < 'AC293', 'F2', 500.50 > into Fare Adds fare options for the newly inserted flights 3. Insert one or more reservations of customers into Reservation table INSERT < 'AC293', 12-Mar-25, 5B, 6:00, 9:30, YYZ, YQT, 'Mark Williams'> into Reservation Perform insert operation to record a customer's seat reservation for the new flight on a specific date and time. No delete or modify operations are needed as all we are doing is adding a new flight. Only needed if any constraints are violated.

b. What types of integrity constraints would you expect to check? (explain using attributes, eg, Ssn of relevant files) 5 marks	Since we are inserting, we must check Domain, Key, Referential integrity and Entity integrity constraints as any of these may be violated. When inserting the flight into the Flight table we must ensure that the Flight# is unique non null and doesn't exist already in the table as Flight# is a primary key and identifies the flight. Then once that is done it is okay to add the Flight# into Fare and reservation tables. When inserting into the reservation table, fltDAport and fltAAport must already exists in the Airport table and be non-null. When inserting into the Fare and reservation tables, we must also make sure that th eFlight# is non null as it is referencing the Flight table We must ensure that the farecode is within the domain for farecodes of {F1, F2, F3}. Fltdate should be valid DD-MMM-YY format and fltDtime must be before fltAtime as you cannot arrive before you depart. Flight# should be ACDDD where DDD is a 3-digit code and AC corresponds to Air Canada
c. Which of these integrity constraints are key, entity integrity, and referential integrity constraints and which are not? 5 marks	Entity Integrity: This constraint ensures that the primaey keys of a tuple are not null. For our insertions, Flight# attribute, primary key cannot be NULL as previously mentioned above. We must also make sure farecode is not NULL as farecode is a part of the primary keys of Fare table. Flight#, Fltdate, Seat#, fltDtime, fltAtime must all be non-NULL as they make up the primary keys of Reservation table. Key constraint: Since we are introducing a new flight# we must make sure the primary key does Flight# does not exist within previous Flight table records. The combination of Flight# and other attributes creates a superkey for Fare and reservation which is why flight# must be unique as it will uniquely identify each tuple from those tables.

	Referential Integrity Constraint: WE must ensure the foreign keys lead to primary keys that exist from another table. For example, the foreign key Flight# from tables Fare and Reservation references the Flight# of the Flight table. So these values must be the same within all the newly introduced tuples for those 3 tables. This guarantees the reservations, and fare exists for the newly created Flight. Domain Constraint: -Farecodes - Fltdate -FltAtime - fltDtime - Flight# should all be within their respective domain of values as written above
d. Specify all the referential integrity constraints on this database. 5 marks	We will write a referential integrity constraint as R.A --> S.A (or R.(X) --> T.A) whenever attribute A (or the set of attributes X) of relation R form a foreign key that references the primary key of relation S (or T). Fare.Flight# -> Flight.Flight# Every fare matches with the new flight Reservation.Flight# -> Flight.Flight# Every new reservation is for the newly introduced Flight Reservation.fltDAport -> Airport.AirportCode Reservation.fltAAport -> Airport.AirportCode Ensures that the Destination and arrival airports do exists

2. (total marks 10) Using your own flight_seat_Reservation database instance from assignment 1, login to the SQL query processor on our cs server, called Oracle Sqlplus to create the four database tables and insert the tuples in your database state with the following sequence of instructions. Note that this exercise is to get you beginning to connect to SQLplus while preparing to learn full SQL language syntax in Chapters 6 and 7. You will be given the instructions to use now. Show the result of this exercise through a Unix script file you will attach as a .txt file.

(Total for que 2 is 10 marks)

i. First connect to our delta.cs.uwindsor.ca through either Bitvise SSH client or NoMachine.
ii. Then hand in a Unix script file to capture your Unix session when you connect to Sqlplus after your instructions for creating your database are working. You can create your Unix script file using the following sequence of instructions on a Unix terminal on our cs server. You need to transfer this script file to your personal computer using a file transfer protocol (eg. Bitvise SFTP or Filezilla) in order to attach it in your assignment submission.

```
>script   username_assn2que2.txt
>sqlplus <username>
>password

SQL> CREATE TABLE AIRPORT
  (AirportCode CHAR(3) NOT NULL,
  Name VARCHAR2(25),
  City VARCHAR2(15),
  State CHAR(3),
  PRIMARY KEY(AirportCode));

SQL>
SQL> CREATE TABLE FLIGHT
  (Flight# VARCHAR2(5) NOT NULL,
 Airline VARCHAR2(15),
 PRIMARY KEY(Flight#));

SQL>
SQL> CREATE TABLE FARE
 (Flight# VARCHAR2(5) NOT NULL,
  Farecode VARCHAR2(2),
 Amount NUMBER(6,2),
 PRIMARY KEY(Flight#,  Farecode),
 FOREIGN KEY (Flight#) REFERENCES FLIGHT(Flight#)
```

```
  ON DELETE CASCADE);

SQL>
SQL> CREATE TABLE RESERVATION
   (Flight# VARCHAR2(5) NOT NULL,
  Fldate DATE NOT NULL,
  Seat# VARCHAR2(3),
  FltDtime NUMBER(6,2),
  FltAtime NUMBER(6,2),
  FltDAport CHAR(3),
  FltAAport CHAR(3),
  Cname  VARCHAR2(25),
  PRIMARY KEY(Flight#, FlDate, Seat#, FltDtime, FltAtime),
 FOREIGN KEY (Flight#) REFERENCES FLIGHT(Flight#)
 ON DELETE CASCADE);

SQL> -- A sample insert of a record into each of the 4 tables is given below
SQL>  INSERT INTO AIRPORT
    VALUES ('YTZ', 'Island Airport', 'Toronto', 'ON');
SQL> INSERT INTO FLIGHT
    VALUES ('WJ250', 'WestJet');
SQL> INSERT INTO FARE
    VALUES ('WJ250', 'F1', 300);
SQL> INSERT INTO RESERVATION
    VALUES ('WJ250', '15-JAN-25', '20A', 8.00, 8.50, 'YQG', 'YYZ', 'Mariane Mooer');

SQL> COMMIT;

// Repeat similar INSERT instructions for all the data in all your tables
// starting with the entity tables first, eg, Airport, Flight, Fare before RESERVATION.

SQL> select * from cat;   // to show all the objects in your catalogue

SQL> select * from Airport;  // to show the contents of this table

SQL> exit                    //to exit sqlplus


>exit                          // to exit and create Unix script file to hand in
```

** More Hint: While in Sqlplus, if you want to delete data from your tables and drop them before issuing your instructions for creating your Unix script file for handing in, you can use the following instructions for each table to first delete the data from the table and then drop the table.

```
delete from RESERVATION;
delete from FARE;
delete from FLIGHT;
delete from AIRPORT;
```

commit;

drop table RESERVATION cascade constraints;

drop table FARE cascade constraints;

drop table FLIGHT cascade constraints;

drop table AIRPORT cascade constraints

commit;
***

Also Note: you can start creating a script file only after you have created your tables correctly and inserted data in the tables. In that case, you cannot re-create existing tables. Then, you can just run the desc table (eg. Desc Person) command for each table to show the structure of each table before using (for example), the (select * from Person;) to show the tuples of each table or delete data and drop the tables as explained above so you can re-create the tables more correctly.

Solution: (10 marks)

An attached Unix script file showing execution of CREATE TABLE instructions and INSERT INTO tablename VALUES instructions with the few SELECT instructions to show contents of the catalogue and all tables (your database instance).

CHAPTER 14: Database Design Theory: Introduction to Normalization Using Functional and Multivalued Dependencies

3. (total marks 10) Consider the following relation:

Enrolled(Studid, Crsid, SName, Score, Lettergrade)

Assume that a student (Studid) may be enrolled in multiple courses (Crsid) and hence {Studid, Crsid } is the primary key.
Thus, the following functional dependency exists:

{Studid, Crsid } -> {SName, Score, Lettergrade}

Additional dependencies in this Enrolled table are:

Studid -> SName

Score -> Lettergrade

Based on the given primary key,

i. is this Enrolled relation in 1NF, 2NF, or 3NF? Why or why not?
ii. If not in 2NF at least, normalize it completely into 2NF and 3NF? Provide your answers using functional dependencies (FDs).

(Total for que 3 is 10 marks)

Solution (i): (5 marks)

Answer:
Given the relation schema
Enrolled (Studid, Crsid, Ctitle, Score, Lettergrade)

with the functional dependencies
{Studid, Crsid } -> {SName, Score, Lettergrade}
Studid -> SName
Score -> Lettergrade
(i) is this relation in 1NF, 2NF, or 3NF? Why or why not?

A relation is in **1NF** if it has a well-defined primary key and all its attributes contain atomic values.
Since the relation does not have any repeating groups or multi-valued attributes, it is in **1NF**.

A relation is in **2NF** if it is in 1NF and **all non-key attributes are fully functionally dependent** on the whole primary key.
Here, {Studid, Crsid} is the **primary key**, but:

Studid → SName violates 2NF because SName depends only on Studid, not on the full composite key.

Score → Lettergrade is a partial dependency because Lettergrade depends only on Score, not on {Studid, Crsid}.

Since there are partial dependencies, the relation **is not in 2NF**.

A relation is in **3NF** if it is in 2NF and has no **transitive dependencies** (i.e., non-key attributes should not depend on other non-key attributes).

Since its not in 2NF, it cannot be in 3NF.

Therefore the given relation is 1NF but not in 2NF or 3NF

Solution (ii) (5 marks)

(ii) If not in 2NF at least, normalize it completely into 2NF and 3NF? Provide your answers using functional dependencies (FDs).

Convert to 2NF: Remove partial dependencies by creating separate tables.

Decomposition:

a. **Student Table (Studid, SName)**
   Studid → SName

b. **Enrolled Table (Studid, Crsid, Score)**
   {Studid, Crsid} → Score

c. **Score Table (Score, Lettergrade)**
   Score → Lettergrade

New relations created:

Student(Studid, SName)
Enrolled(Studid, Crsid, Score)
Score_Letter(Score, Lettergrade)

Convert to 3NF: Remove transitive dependency (Score → Lettergrade).

The relation Score_Letter(Score, Lettergrade) already isolates the transitive dependency.
Now, all relations are in **3NF** since all attributes are only dependent on keys.

4. (total marks 10) What (i) update, (ii) delete and (iii) insertion anomalies occur in the DEPARTMENT_PROJECT relation obtained by doing a natural join of the two relations DEPARTMENT and PROJECT of Fig 14.2 on page 463 of book? Explain with examples using this database and the DEPARTMENT_PROJECT relation schema with state given below as Figures 4.1 and 4.2 below.

(Total for que 4 is 10 marks)

Note: 3 marks for correct discussion of each anomaly and 1 mark for attempt.

Figure 14.2 (book): Sample database state for a simplified COMPANY relation DB

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

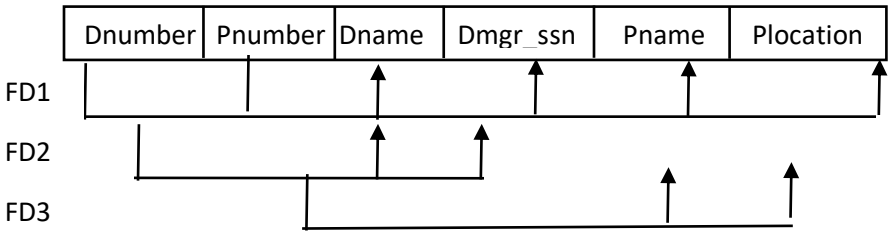
Ssn	Pnumber	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	Null

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

****

Fig 4.1: DEPARTMENT_PROJECT DB schema suffering from update anomalies



***

Fig 4.2: A database state of the DEPARTMENT_PROJECT DATABASE derived from Fig 14.2

DNUMBER	PNUMBER	DNAME	DMGR_SSN	PNAME	PLOCATION
5	3	Research	333445555	ProductZ	Houston
5	10	Research	333445555	Computerize	Stafford
5	20	Research	333445555	Reorganize	Houston
5	30	Research	333445555	Nbenefits	Stafford
5	1	Research	333445555	ProductX	Bellair
5	2	Research	333445555	ProductY	Sugarland
4	3	Administration	987654321	ProductZ	Houston
4	10	Administration	987654321	Computerize	Stafford
4	20	Administration	987654321	Reorganize	Houston
4	30	Administration	987654321	Nbenefits	Stafford
4	1	Administration	987654321	ProductX	Bellair
4	2	Administration	987654321	ProductY	Sugarland
1	3	Headquarters	888665555	ProductZ	Houston
1	10	Headquarters	888665555	Computerize	Stafford
1	20	Headquarters	888665555	Reorganize	Houston
1	30	Headquarters	888665555	Nbenefits	Stafford
1	1	Headquarters	888665555	ProductX	Bellair
1	2	Headquarters	888665555	ProductY	Sugarland

18 rows selected.

Solution: (3 + 3 + 3 + 1 marks)

The DEPARTMENT_PROJECT relation is obtained by doing a natural join between DEPARTMENT and PROJECT, which can cause anomalies.

i. Update Anomalies:

Since **DMGR_SSN** (Department Manager SSN) is repeated for each project within a department, any update to the department manager's SSN must be made in **multiple rows**.

Ex:

Suppose Research department's manager changes from 333445555 to 444556666, this change must be updated in **all rows** where DNAME = Research.

If we forget to update even one row, it leads to **inconsistent data**. This is an update anomaly.

ii. Delete Anomalies:

If we delete a **department's last project**, we lose all information about the department.

Ex:
Suppose **Headquarters** only has one project ProductZ, and we delete this project.
The department **Headquarters** itself is lost since it does not exist independently in another relation.

iii. Insertion anomaly:

If a **new department** is created but has no projects yet, we **cannot insert it** into DEPARTMENT_PROJECT without creating a dummy project.

Ex:
If we want to add a new department **"Finance"**, but it has no projects yet, we **cannot insert** it since PNUMBER (Project Number) is a required attribute.

iv. Solution to prevent these anomalies:

To prevent these anomalies, we should **decompose** the relation into two separate tables:

Department(DNUMBER, DNAME, DMGR_SSN)
Project(PNUMBER, PNAME, PLOCATION, DNUMBER)
WORKS_ON(SSN,PNUMBER, HOURS)

This decomposition **eliminates redundancy** and prevents update, delete, and insertion anomalies.